课程笔记：规划、任务分解与多 agent 协作

Bowen-mantou（整理）

2026-10-23

视频作者/频道：B 站「常青藤名校课」· CMU 11-768 AI Agents（英文原声字幕版）
发布日期：2026-09-09
视频时长：75:14
视频链接：https://www.bilibili.com/video/BV1a6Yx62EH4?p=19
PDF 制作 Skill：https://github.com/wdkns/wdkns-skills

目录

1 引言：为什么 agent 需要规划 **2**

1.1 一个真实的长任务：帮实验室采购 GPU 2

1.2 任务里本来就有结构：依赖图与重规划 2

1.3 本讲的六个问题 2

1.4 什么是计划：本讲的工作定义 3

1.5 本章小结 3

2 从思维链到任务分解：规划的前史 **3**

2.1 思维链：一次生成里的「计划 + 解题」 3

2.2 任务分解：先拆题，再逐题解 4

2.3 为什么这套东西到了 agent 场景会不够用 4

2.4 本章小结 4

3 经典规划与 LLM 规划：显式与隐式 **4**

3.1 经典规划：状态、动作、前置条件与效果 5

3.2 LLM 规划：一切都在隐式知识里 5

3.3 本章小结 5

4 计划怎么表示：形式化、自然语言与程序 **6**

4.1 本章小结 6

5 结构化的谱系：从固定工作流到自适应 agent **6**

5.1 三种形态 6

5.2 交易是什么 7

5.3 plan mode：人机共同编辑计划 7

5.4 本章小结 7

6 模块化：把规划器和执行器分开 **7**

6.1 一个具体案例：会规划但不会点鼠标的模型 7

6.2 也可以训练一个规划器 8

6.3 模块化的代价 8

6.4 本章小结 8

7 执行反馈：重规划、交互与过度思考 **8**

7.1 来自环境的失败反馈：AlfWorld 的例子 9

7.2 把「能不能执行」也纳入规划：SayCan 9

7.3 想得更多 vs 做得更多 9

7.4 过度思考：规划太多也会伤表现 9

7.5 本章小结 10

8 长程任务：误差乘法、上下文中毒与注意力 **10**

8.1 Vending-Bench：经营 222 天的自动售货机 10

8.2 精度乘法：为什么 90% 远远不够 10

8.3 上下文中毒：过去的错误让未来的错误更可能 10
8.4 本章小结 11

9 可训练的规划：RAO（递归智能体优化） 11
9.1 把「委托」变成一个动作 11
9.2 奖励怎么分解 11
9.3 本章小结 12

10 人机接口与安全：计划作为边界与介入点 12
10.1 计划作为安全边界：把不信任的信息挡在计划之外 12
10.2 人如何编辑与批准计划：Cursor plan mode 演示 12
10.3 长程电脑操作的评测：Odysseus 13
10.4 本章小结 13

11 多 agent：管理者 + 子代理的结构与实证 13
11.1 MACU 的结构：DAG + 调度器 + 重规划 13
11.2 实验证据：多 agent 在哪里有用 13
11.3 这对系统设计意味着什么 14
11.4 本章小结 14

12 总结与延伸 14
12.1 主讲人的收尾 14
12.2 把本讲压缩成六句话 15
12.3 延伸思考 15
12.4 拓展阅读（课程中提到的文献，供进一步查阅） 16

1 引言：为什么 agent 需要规划

本讲讨论规划（planning）与任务分解（task decomposition）。这两个词听起来抽象，但主讲人一上来就把它锚定在一个具体到不能再具体的任务上，让整讲的问题意识非常清楚：**当 agent 面对一个需要几百步、跨越多个信息源、而且中途情况会变的真实任务时，它应该怎么组织自己的行为**。

1.1 一个真实的长任务：帮实验室采购 GPU

任务是这样的：为实验室买一台新的 GPU 工作站，需要比较不同厂商的配置与价格，对照内部 wiki 上的集群约束检查兼容性，最后提交采购申请。这样一个任务，如果用一个标准的 ReAct 循环（边想边做、每步看环境反馈）去做，**大概要几百步**：先去厂商 A 的网站、导航到工作站页面、找到某个配置、抄下参数；再回到 wiki 对照约束；然后对 B、C、D 重复同样的流程——而且「到底要看几个厂商」这件事本身，一开始也不确定。

图 **??** 把这种执行过程摊开了：第 1 步打开厂商网站，第 15 步在配置 2 张 RTX 6000、128GB 内存、850W 电源，第 31 步在 wiki 读到「机架接受 4U 机箱、每个插槽 1600W」，中间甚至发生了上下文压缩（128K 压到 14K token）。到第 201 步，agent 面对的是一个真实的僵局：四家厂商都看过了，只有 D 满足规格，但交期要 20 周。这时候它只有三个选项：认了、问人、或者再找第五家。

这个例子把本讲的核心张力摆了出来：**一个只会「走一步看一步」的 agent，在长任务里会迷失方向；但如果一开始就把所有步骤写死，它又无法应对环境里冒出来的新信息**。规划就是在这两端之间做设计。

1.2 任务里本来就有结构：依赖图与重规划

主讲人接着把同一件事画成了一张图，也就是图 **??**。这张图是理解整讲的钥匙：厂商 A、B、C、D 四家的调研彼此独立，可以并行；四家的信息汇总之后，才能做兼容性检查；兼容性检查通过之后，才能提交采购申请。可见任务本身存在一个**依赖图**（dependency graph），而不是一条直线。

更有意思的是：如果四家都不合适，就需要**新增节点**——再找第五家厂商、或者放松某项规格、或者去问人。这种「在执行过程中修改计划」的行为，就叫**重规划**（replanning）。所以计划不是一次性写死的剧本，而是一个**可以随观测更新、可以增删节点的结构化对象**。

1.3 本讲的六个问题

沿着这个例子，主讲人列出了本讲要回答的问题，它们也构成本讲后续各节的主线：

1. 让 agent 明确承诺「我接下来要做什么」，有用吗？什么时候有用，什么时候不值得付出这份额外的成本与刚性？
2. 计划应该怎么表示？自然语言？结构化的图？还是代码？
3. 如果做了计划，什么时候应该改变它？比如要不要去查第五家厂商？
4. 人应该在什么时候、以什么方式介入？
5. 怎么把「规划能力」本身训练进模型？
6. 多 agent（管理者 + 子代理）能带来什么，代价是什么？

1.4 什么是计划：本讲的工作定义

主讲人给了一个可以直接背下来的定义：**计划是对智能体未来行为的显式表示（explicit representation of intended future behavior）**——包括它将要尝试的动作或子目标，以及这些动作/子目标之间的顺序与依赖关系。

本讲的工作定义：计划（plan）

计划 = 对 agent 未来行为的显式表示：要做的动作或子目标，加上它们之间的顺序（ordering）与依赖（dependencies）。计划是一个**提案**（proposal），不是契约；agent 可以并且应该根据环境中的新信息偏离、修改它。

这里要和上一讲的「技能」（skill）做个区分：技能是**跨任务可复用的指导**，而计划只服务于**当前这一个任务**，因此可以更具体——甚至可以绑定到某个具体的代码仓库。另外，主讲人引用了一句拳击手 Mike Tyson 的名言：「每个人都有一个计划，直到脸上挨了一拳。」——计划要有，但也要能在挨打之后改。

本讲路线图

规划为什么值得做（动机与依赖图）→ 从思维链到任务分解（规划的前史）→ 经典规划 vs LLM 规划 → 计划的三种表示（形式化/语言/程序）→ 从固定工作流到自适应 agent 的谱系 → 规划器与执行器分离（模块化）→ 执行反馈与重规划（含过度思考）→ 长程任务的误差与上下文中毒 → 可训练的规划（RAO）→ 计划作为安全边界与人机接口 → 多 agent 的实证。

1.5 本章小结

长任务天然带有依赖结构；只靠反应式循环会迷失，而写死的流程又无法适应变化。计划是对未来行为的显式表示（动作/子目标 + 顺序/依赖），本质是提案而非契约，可以在执行中被修改，即重规划。本讲要回答的是：计划何时有用、如何表示、何时修改、人如何介入，以及如何训练与评测。

2 从思维链到任务分解：规划的前史

要理解 LLM agent 的规划，得先理解它的前身——思维链（chain of thought, CoT）。主讲人把这段历史压成了两步：**先是用提示词让模型产生推理过程，再是把复杂问题拆成子问题**。

2.1 思维链：一次生成里的「计划 + 解题」

在模型被专门训练成推理模型之前，研究者发现只要在提示里引导模型「一步一步想」，答案正确率就会提升。最著名的是 Kojima 等人 2022 年的工作：「Let's think step by step」这一句简单的提示词，就让模型的准确率明显上升；后来 Wang 等人（2023）的提示词更进一步，直接要求模型「先理解问题、制定一个计划，再按计划一步步解」。

图 ?? 展示了这两种提示词，以及幻灯片上总结的三点：第一，思维链为模型提供了**额外的计算量**，也提供了一个可以回头看的**草稿纸**（scratchpad）；第二，**计划和解答是在同一次生成里一起出来的，没有任何外部组件显式检查过这个计划**；第三，思维链可以被训练，让正确作答的概率更高（STaR、Zelikman 等人的工作，以及 DeepSeek R1 这类推理模型）。

这里有一个值得记住的工程判断：为什么思维链有效？主讲人给了两个直觉。一是**算力**——把中间结果写进 token 里，等于让模型在预测最终答案前多做若干步运算；二是**外部化**——把解题过程从「脑内」搬到「纸上」，后面每一步都能引用前面的中间结果。这两点在 agent 规划里会以同样的理由再次出现。

2.2 任务分解：先拆题，再逐题解

沿着思维链，自然的一步是：**与其让模型一口气解复杂问题，不如先让它把问题拆成子问题，再逐个解**。图 **??** 展示了这个两阶段流程（least-to-most prompting 一类方法）：第一阶段，语言模型把复杂问题分解成若干子问题；第二阶段，依次回答这些子问题，并且把复杂问题本身和前面子问题的答案一起作为输入，传给下一个子问题。

幻灯片右侧把好处写得很清楚：子问题比原问题更容易回答；**回答子问题只需要更少的上下文**（要算「每趟多久」时，不需要同时想着「滑梯 15 分钟后关闭」）；而且随着问题变长，这种分解带来的提升会更明显。

同一个思路还有一个变体：**decompose prompting**。它不只生成子问题，还会给每个子问题**预测一个类型**（比如一般问答、关于人物职业的关系型问答），不同类型的子问题可以用不同的提示词、甚至不同的专门模型来处理。它还带了一个简化版的「程序化控制」：显式指定子任务的执行顺序，并允许把前一个子任务的答案作为参数传给下一个。

讲述到这里，主讲人补了一个很重要的观察：这和上一讲的「从模板到代码」是同一条演化路径——一开始用结构化的领域特定语言描述子任务编排，后来直接**用代码来写**这种编排，即「计划即程序」。这条线在后面的 RAO 里会走到极致。

2.3 为什么这套东西到了 agent 场景会不够用

思维链和任务分解都是在「生成文本」：推理步骤只是文字，不会真的改变外部世界，也不会揭示模型原本不知道的信息。而 agent 一旦开始行动，就会遇到三件推理模型从未面对的事情：

1. **动作会揭示新信息**：去厂商网站调研，才会知道某型号交期 20 周；
2. **动作会失败**：计划写得很好，但要用的东西当前拿不到；
3. **动作可能不可逆**：删掉文件、提交订单、发出邮件。

因此，agent 的规划必须把「执行中获得的反馈」当成一等公民——这正是后文「重规划」一节的主题。

2.4 本章小结

思维链让模型用额外算力与草稿纸提升解答质量，但计划与解答在一次生成里完成、无人检查；任务分解进一步把复杂问题拆成子问题、逐个求解，好处是子问题更简单、所需上下文更少，还可以给不同子任务配不同模型。agent 场景的复杂性在于动作会揭示信息、会失败、可能不可逆，因此计划必须可修订。

3 经典规划与 LLM 规划：显式与隐式

在 LLM 之前，AI 里已经有一整套成熟的规划研究。主讲人用一页幻灯片（图 **??**）把它讲清楚，然后用它和 LLM 规划做对照。

3.1 经典规划：状态、动作、前置条件与效果

经典规划的核心是**用逻辑化的方式表示世界**。世界状态（state）是一组谓词（predicates）的集合，比如「方块 x 在桌上」「方块 x 在方块 y 上」「手是空的」；动作（actions）是对这些状态的更新函数，每个动作都有**前置条件**（preconditions，什么时候允许执行）和**效果**（effects，执行之后世界变成什么样）。

幻灯片用的例子是积木世界：`pick-up(x)` 的前提是「x 在桌上、x 上面是空的、手是空的」，效果是「拿起了 x、x 不在桌上、手不再空」。一个规划问题由初始状态和目标状态组成，**规划就是找一条从初始状态到目标状态的动作序列，使得序列里每个动作的前置条件都被当时的状态满足**。这类表示法最著名的形式是 STRIPS，后来的 PDDL 也被用来当作 LLM 规划的评测目标。

大量经典规划研究其实是在做**搜索**：给出这套高度结构化的表示，用 BFS、DFS、A* 以及各种启发式函数，找出从起点到终点的动作序列。它的好处是每一步都可以被形式化验证，计划是可检查的。

3.2 LLM 规划：一切都在隐式知识里

换成 LLM，情况几乎全反过来了。对 LLM 来说，**一切都是隐式的**：它不知道某个动作确切的前置条件是什么，也不完全知道动作的后果；这些知识散落在外部世界里，模型只能「先试试看」。但反过来，LLM 有一个经典规划器没有的优势：**它不需要做很多搜索就能写出一个看起来合理的计划**——因为它在预训练中见过大量类似的场景，可以靠先验直接生成「先把积木叠起来」或「先看日志再改代码」这样的计划。

所以 LLM 时代规划的重心变了：**从「搜索出计划」变成了「检查计划是否可行、是否可执行」**。但经典规划里的两个概念依然重要——搜索（后来在 test-time search 里回归）和动作的适用性（action applicability，即当前状态能不能执行这个动作）。

表 1: 经典规划与 LLM 规划的对照

维度	经典规划（STRIPS/PDDL）	LLM 规划
世界表示	显式：谓词、状态、动作模型	隐式：在模型参数与环境交互中
动作前提	显式定义、可形式化检查	不明写，靠试错与环境反馈
计划生成	搜索（BFS/DFS/A*+ 启发式）	直接生成（先验强），再做可行性检查
计划正确性	可验证：每个前置条件都满足	形式化不可验证，只能执行后看结果
难点	搜索复杂度、状态空间爆炸	可行性判断、反馈后的重规划、不可逆动作的风险

3.3 本章小结

经典规划用显式的状态/动作/前置条件表示世界，把规划变成搜索；LLM 规划把这些都藏在参数里，靠先验直接生成计划，重心从「搜索」转向「检查可行性」。两个概念仍然重要：搜索（test-time search）与动作适用性（affordance）。

4 计划怎么表示：形式化、自然语言与程序

既然不做经典规划，那 LLM agent 的计划到底长什么样？主讲人给出三种表示，并强调它们是一个谱系：**形式化的计划可以检查；自然语言的计划最通用；程序介于两者之间**。

1. **形式化计划**（如 STRIPS 的谓词序列）：可检查，但要求世界被完整形式化，对开放环境不现实。
2. **自然语言计划**（如 plan mode 里那份 markdown 待办列表）：通用、可读、可被人编辑，但不可形式化验证。
3. **程序即计划**（code as plan）：2023 年前后出现的一条工作线——用代码模型写一段程序，程序里调用感知 API 与控制 API，由程序来「执行计划」。

图 **??** 的机器人例子很直观：用户说「把积木堆到空碗上」，模型生成一段 `Policy Code`：先 `detect_objects` 找到所有积木与碗，再循环找空碗，最后调用 `stack_objects` 把积木逐个叠上去。幻灯片总结道：**形式化计划可检查，语言计划通用，程序介于两者之间**；程序可以被运行、被检查，但受限于它可用的 API 与程序结构；而这些 API 内部**也可以调用 LLM 或神经网络模型**——于是你既得到了代码的结构化控制，又保留了学习模型的柔性。

主讲人还提到，这类「代码即计划」在机器人领域后来没有像端到端的多模态 VLA 系统那样流行，但对**编码 agent** 非常有价值——这正是下一讲 CodeAct 的伏笔。代码作为计划还有两个具体好处：一是**可以表达重试与后置条件**（「积木没放上去？那就再试一次」），二是执行结果可以被人审计。代价是它比自然语言更刚性——程序写不出来的事情就做不了。

4.1 本章小结

计划有三种表示：形式化（可检查但要求世界被形式化）、自然语言（通用、可被人编辑）、程序（介于两者之间，可运行可审计，但受 API 与结构限制；API 内部可调用学习模型）。选择哪种表示，取决于你需要多强的可检查性、多强的通用性，以及谁来读这个计划。

5 结构化的谱系：从固定工作流到自适应 agent

计划的第一大价值是**用结构换取可预测性**。主讲人借用 Anthropic 那篇「Building Effective Agents」博客的观点，把系统放在一个谱系上：最左端是开发者写死的流程，最右端是完全自适应、几乎不预先规划的 agent。

5.1 三种形态

图 **??** 中间那条从左到右的箭头，表示决策发生在执行前（decisions made before execution）还是执行中（decisions made during execution）：

1. **开发者定义的工作流**（developer-defined workflow）：开发者写死流程，比如「永远调研这四家厂商，读规格、检查兼容性、选最便宜的买」。图里还给了对应的伪代码：`for v in VENDORS: specs[v] = read_specs(v)`，然后筛出兼容的、买最便宜的。下一讲要讲的 agentless 方法就是这类「为某个问题高度定制的工作流」。
2. **先规划再执行**（plan-then-execute agent）：模型先决定「要检查哪些厂商」，在执行前就把计划定下来，然后按计划执行。

3. **自适应 agent**（adaptive agent）：如果四家都失败，执行反馈会让模型再去找第五家。RAO 与 MACU 都属于这一类。

5.2 交易是什么

幻灯片右下角把权衡写得很直接：**提前承诺计划**更快、更便宜、更可预测（因为可以让更小、更便宜的模型去执行预先拆好的子任务）；但如果完全不改，就**失去了适应环境变化的能力**。而重规划有时值得它带来的额外成本、复杂性与不可预测性。

主讲人还补充了一个历史判断：Anthropic 后来把这篇博客标注为过时，部分原因是现在的 agent 本身变强了——最新的模型在不受工作流约束时往往也能做出合理的动作；在 coding agent 上也能看到同样的趋势，agentless 一类固定工作流不再是解决问题的最佳方案。但另一方面，如果任务是新领域、模型还没被训练得能应对环境变化，**从工作流开始仍然是更安全的选择**。

5.3 plan mode：人机共同编辑计划

现代 coding agent 的 plan mode 是这套谱系在工程上的一个漂亮落点（后面第 10 节还会结合 Cursor 的演示细讲）：用户给出复杂任务，agent 先做**非破坏性的信息收集**（读代码库、搜索文件），然后产出一份自然语言/ Markdown 的计划；人审核、直接编辑计划或提反馈；确认之后点「build」，agent 才开始执行，且可以把计划里的条目分给子代理并行执行。

主讲人现场问了两个问题：「多少人用过 coding agent 的 plan mode?」（不少人举手）和「多少人敢让 agent 直接替你买 GPU?」（几乎没人举手）。这两个回答恰好说明了计划的价值不是「让 agent 更聪明」，而是**让人能在风险变高之前介入**。

5.4 本章小结

结构化谱系从「开发者写死的工作流」到「先规划再执行」再到「自适应 agent」：越往左越便宜、可预测，越往右越能适应变化。提前承诺计划有明显收益，但会牺牲适应性；重规划值得它带来的成本时才该用。plan mode 是这条谱系在工程上的经典落点——计划是人机共同编辑的对象，也是人介入的时机。

6 模块化：把规划器和执行器分开

计划的第二大价值是**模块化**：如果能把任务分解成子任务，就可以给不同的子任务配不同的模型、不同的训练数据，各自独立优化。

6.1 一个具体案例：会规划但不会点鼠标的模型

图 **??** 展示的是 2025 年一波 web agent 的典型结构。当时最强的模型（比如 GPT-4o 级别）是「视觉 grounded」的——它看得懂网页，也能用自然语言描述下一步该做什么，比如「点页面顶部的搜索框，输入 4k monitor」；但它**做不好像素级动作**，尤其在需要精确点击某个坐标时。

解决办法是把这两件事拆开：让强模型负责**规划**（把观察翻译成自然语言动作描述），再训练一个 **7B 的 grounding 模型**，把「页面顶部那个搜索框」这样的描述映射到具体屏幕坐标 `(556, 26)`，最后由执行器点击、输入。幻灯片上方那一排小标签（Modularity / Feedback / Long horizon / Control）是整讲的四个主线；Modularity 这一栏下写着「decompose and specialize」。

这种拆分带来两个直接好处：第一，可以**独立优化每个组件**——先测出瓶颈在哪（是规划差还是 grounding 差），再把算力/数据集中投到瓶颈上，一个组件一个组件地「爬坡」；第二，可以**用更便宜的模型执行**，把贵模型的调用限制在真正需要强推理的规划步骤上。

6.2 也可以训练一个规划器

除了训练执行器，也可以反过来生成数据来训练规划器。主讲人介绍了 Plan-and-Act 这类做法：先让模型在环境里跑出大量完整轨迹，**筛选出判定为成功的轨迹**；再让一个教师 LLM 对这些轨迹做**分段并给每段写自然语言标签**；把分段结果「解压」成训练数据，就得到了「任务描述 → 步骤序列」的规划器训练样本，以及「步骤描述 → 具体动作」的执行器训练样本。

这其实和上一讲「工作流归纳」的思路同源：都是从轨迹出发，做分段与标注；区别是这里的分段是为了训练一个能生成计划的模型。

6.3 模块化的代价

主讲人没有回避这套思路的弱点。把任务分解成子任务再串起来，是一层**抽象**，抽象会丢掉细节、也会变得不灵活。几个具体代价：

1. **角色分工不一定匹配任务**。有一批工作给 coding agent 设定「软件公司」式的角色分工（测试、编辑，甚至产品经理），其中一些变体（如 agentless）在「任务总是匹配这套结构」时很成功；但当任务不匹配时，它们缺乏适应性。
2. **模块之间要传信息，而传对信息很难**。上一个模块的输出未必是下一个模块需要的；如果换成单 agent 且上下文管理做得好，过去所有动作的信息本来都在那里，可以直接用。
3. **固定角色会锁死任务流**。图 ?? 之外还有一页专门提醒（「Beware of fixed roles and multi-agent systems」）：角色是任务开始前就固定的，某个角色卡住时其他 agent 可能还在等；交接靠摘要报告，而摘要可能丢掉下一棒真正需要的细节。

模块化的常见误区

「拆成多 agent 就一定更好」是错的。拆分会引入信息传递损失与固定结构的不灵活；只有当子任务确实可分离、且每个子任务值得用专门的模型或数据去优化时，模块化才划算。否则一个上下文管理良好的单 agent 反而更强。

6.4 本章小结

模块化通过分解换取可优化性：planning 与 grounding/execution 可以分别用不同模型、不同数据，先找瓶颈再逐个优化。代价是抽象丢细节、固定角色不灵活、模块间信息传递困难；因此模块化是手段而不是目的。

7 执行反馈：重规划、交互与过度思考

计划的第三大价值是**在执行中适应**。这一节把「计划—执行—反馈—重规划」的闭环讲透，并给出两个反向提醒：不是想得越多越好，过度规划会伤害表现。

7.1 来自环境的失败反馈：AlfWorld 的例子

图 **??** 是一个具身环境（AlfWorld）中的例子：任务是「煮一个土豆并放进回收箱」。规划器先生成一个高层计划（导航到土豆、拿起、放进回收箱……），交给机器人执行；机器人反馈「我找不到土豆，但我看到了冰箱」；规划器据此重规划：先打开冰箱、再拿土豆；之后又收到「找不到回收箱，但看到了垃圾桶」，于是再次调整计划。

这就是**基于环境反馈的重规划**：把环境的自然语言反馈当作条件，重新生成计划。它和上一讲提到的 reflection（反思）是同一条线，但更强调**计划对象本身在变**，而不只是重试。

7.2 把「能不能执行」也纳入规划：SayCan

经典规划有前置条件检查，LLM 规划没有。SayCan 的思路是把这个检查补回来：既然 LLM 会生成一堆「看起来合理」的子任务，那就再训练一个**价值函数**（value function），估计在当前状态下执行某个子任务的成功概率，也就是 **affordance**（可供性）。

图 **??** 展示了这个组合：左边是 LLM 对候选子任务的打分（「find an apple」、「find a coke」等），中间按可执行概率重排，右边是机器人当前视角；最终把两个概率**相乘**：

$$\text{score}(\text{subtask} \mid \text{context}) = p_{\text{LM}}(\text{subtask} \mid \text{instruction}) \times p_{\text{affordance}}(\text{subtask} \mid \text{state}).$$

其中 p_{LM} 是语言模型给这个子任务的概率（它由提示词示例、上下文决定），$p_{\text{affordance}}$ 是价值函数估计出的、在当前画面/状态下成功执行该子任务的概率。二者相乘，相当于「语言上合理」且「物理上可行」才会被选中。这就是经典规划里「动作适用性」的 LLM 版本。

7.3 想得更多 vs 做得更多

既然计算预算有限，就有一个分配问题：**同样多的 token，应该花在「想」上，还是花在「做」上**？图 **??** 是一个受控实验：匹配三种推理方法的 token 总量，横轴是每条轨迹的 token 数（对数），纵轴是任务成功率。三条线分别是：每步预算强制（Per-Step Budget Forcing，蓝）、每步 best-of-N（橙）、交互扩展（Interaction Scaling，绿）。

结论很明确：**把同样的 token 花在与环境交互上，通常比花在更长的思维链上更划算**——绿线在相同 token 预算下成功率更高。幻灯片还引用了一个具体的交互技巧：「你刚刚宣布任务完成？暂停一下再想想。」这就是 Reflexion 式的自我反思；而交互扩展获得的是环境信息（比如看到新页面），这比模型自己空想出来的信息更可靠。

7.4 过度思考：规划太多也会伤表现

反过来，规划太多也确实会出问题。主讲人（这项工作他本人也参与）把推理模型在 agent 场景里的坏行为归成三类：

1. **分析瘫痪**（analysis paralysis）：在规划上花太多时间/ token，真正执行的动作太少；
2. **有害动作链**（rogue action chains）：做了一系列没有用、甚至破坏环境的动作；
3. **过早结束**（premature disengagement）：任务还没做完就宣布完成。

关键在于：这些行为**不需要读取推理轨迹**就能识别——只需要知道模型花了多少推理 token、做了多少动作。作者据此训练出分类器，给出一个「过度思考分数」（overthinking score），然后在

SWE-bench 这类编码任务上比较它与解决率的关系。图 **??** 的结果是明显的负相关：推理模型趋势线 $R^2 = 0.892$、斜率 $\beta = -7.894$；非推理模型 $\beta = -15.938$——过度思考越严重，解决的 issue 越少。而且这个现象不限于推理模型：只要模型在动作前产生一些文本，就可能有类似倾向，只是推理模型更严重。

主讲人给出了缓解方向：换成更强/更大的模型、或者降低推理强度（reasoning effort），过度思考会减少、表现会更好。这个结论对做 agent 系统的人非常实用：**不要默认「想得越多越好」，要把「想/做」的配比当成一个需要调的超参数**。

7.5 本章小结

执行反馈让计划可以重规划（AlfWorld 例），而 affordance 模型（SayCan）把「能不能执行」补进了规划打分。计算预算在「想」与「做」之间有明确权衡：相同 token 下多交互通常更优；反过来过度思考（分析瘫痪、有害动作链、过早结束）会显著伤害表现，且可以通过「推理 token 数」这类廉价信号监测。

8 长程任务：误差乘法、上下文中毒与注意力

计划真正难的地方，是在**长程**（long horizon）任务上。主讲人用 Vending-Bench 这个例子把难度量化了。

8.1 Vending-Bench：经营 222 天的自动售货机

图 **??** 里的任务看起来简单：经营一台自动售货机——定价、进货、补货。但它难在**时间跨度**：任务最长要跑 222 个模拟日、约 2,000 条消息、约 2,500 万 token。模型必须长期跟踪状态（库存、现金、价格），并且做出一个又一个不把过去错误延续下去的决定。

幻灯片右侧给了一个很有说服力的失败例子：「提前补货」——把「今天到期」误当成「已经到货」，环境正确地报告「商品不可用」，但模型已经基于这个错误继续做决策了。一句话总结：**一个临时的错误会影响之后的每一个决定**。

8.2 精度乘法：为什么 90% 远远不够

长程任务的第一条数学事实是**概率相乘**：如果每一步做对的概率是 p，要做 n 步连续做对，整条轨迹成功的概率是

$$P(\text{success}) = \prod_{i=1}^{n} p_i \approx p^n.$$

$p = 0.9$ 时，10 步全对只有约 35%，20 步只剩约 12%；而 $p = 0.999$ 时 100 步还有约 90%。幻灯片用两句对比概括这件事：「即使单步准确率从 0.9 提升到 0.999，对能正确执行的任务长度也有巨大影响」；反过来，「更高的单步准确率是长程能力的前提」。这也解释了为什么要把子步骤做简单、把计划做细——每一步都更难，乘法就吃掉一切。

8.3 上下文中毒：过去的错误让未来的错误更可能

第二条事实更隐蔽：模型在生成时依赖过去的上下文，而预训练数据里，**高质量内容往往成片出现**——一个文件里代码质量高，通常整个文件质量都高；一个有 bug 的文件，后面多半也有问题。

于是模型学到一个先验：上下文质量差，后续输出质量也差。这在长任务里变成恶性循环：早期犯的错留在上下文里，会让后面更容易继续犯错（context poisoning）。

主讲人引用了一项对 Qwen3 系列模型的分析：在模型过去的上下文里按不同比例注入错误，然后测准确率——注入错误比例越高，准确率越低；而且**最大的模型 Qwen3-32B 下降反而最明显**，说明强模型并不天然免疫上下文中毒。好消息是强化学习可以很大程度上纠正这一点：RL 训练出的策略更倾向于「从错误中恢复」，而不是被过去的错误带偏。

8.4 本章小结

长程任务同时受两个机制约束：一是精度乘法（p^n），单步精度必须非常接近 1；二是上下文中毒，过去的错误会提高未来犯错的概率，强模型也不免疫。对策是降低单步难度（分解）、限制/压缩上下文、以及用 RL 训练恢复能力。

9 可训练的规划：RAO（递归智能体优化）

前面所有讨论都还是一个「系统设计」问题；到了这里，主讲人转向「能不能把规划训进模型」。代表工作是 RAO（Recursive Agent Optimization，论文作者包括本课助教）。

9.1 把「委托」变成一个动作

RAO 的核心想法可以用一句话概括：**让模型生成一段代码作为计划，并通过子代理（subagent）执行；「委托」本身是一个可训练的动作**。具体来说，模型被训练去写这样的代码：

```
# 计划即代码：用 async/await 控制并行/串行的子代理
cherry = await launch_subagent("find the best cherry blossom spots in Kyoto")
temple = await launch_subagent("find quiet temples in Kyoto")
hotel  = await launch_subagent("find a hotel near the temple")
plan   = await launch_subagent(f"plan a 3-day trip using {cherry}, {temple}, {hotel}")
```

Listing 1: RAO 风格的计划：模型生成控制子代理的代码（示意，非原始实现）

这段代码里有两个关键设计。第一，**子代理是递归的**：子代理本身也可以再调用 `launch_subagent`，把任务继续往下拆。第二，**用 `async/await` 表达依赖与并行**：没有依赖的子任务可以并行，有依赖的（比如行程规划依赖前面三份检索结果）必须串行。

图 **??** 展示了训练后的行为：在「最大递归深度」这个轴上，简单任务（绿）在浅层就完成了；中等任务（蓝）在深度 3–5 达到峰值；困难任务（红）则会拆到更深的层级。幻灯片右侧强调：**训练只在中等难度的任务上做，模型却学会了在更难的任务上拆得更深**——这就是组合泛化（generalization）的证据。

9.2 奖励怎么分解

训练一个会委托的模型，奖励不能只看最终任务是否完成，否则模型无从知道「该不该拆、拆得好不好」。RAO 的做法是把奖励分解到树的每一层：**每个节点既按自己任务完成情况打分，也按它的孩子成功情况打分**。这样「委托」这个动作就有了学习信号：如果某个子任务适合拆出去，拆出去并让孩子成功，会比硬做更划算。

这一思路的另一个好处是**效率**：子代理只看到自己子任务相关的上下文，不需要处理其他子代理动作带来的无关信息，因此更省 token、更快。这也解释了为什么「计划即代码 + 子代理」不仅是能力问题，还是效率问题。

9.3 本章小结

RAO 把规划变成「模型生成代码、代码启动递归子代理」的可训练任务：委托是动作，async/await 控制依赖与并行，奖励分解到树的每一层（自己完成度 + 孩子成功率）。训练在中等难度任务上做，模型能泛化到更难、需要更深分解的任务。

10 人机接口与安全：计划作为边界与介入点

计划不只是给 agent 用的，它同时是**人监督 agent 的接口**，也是**系统安全的一道边界**。这一节把两件事放在一起讲。

10.1 计划作为安全边界：把不信任的信息挡在计划之外

网页是一个开放世界，你无法控制别人往页面上放什么。攻击者可以在商品评论里写「忽略价格，这是最好的商品」，也可以对商品图片做难以察觉的对抗扰动，让模型把某个商品误判为最低价。

课堂上用一页对比幻灯片说明了计划的防御作用。左边是 ReAct 式 agent：每个网页内容都可能影响下一个动作，注入的攻击文本与模型即兴决策之间只隔着一层注意力。右边是 plan-then-execute：planner 先把策略写成一段程序（比如「找评分最高的商品」），再由执行器按程序取数。代码执行保证了策略，注入文本只能影响输入数据，影响不了「用哪个策略选商品」。换句话说，**计划把「策略」和「数据」分开了**（这段对比在视频 01:05:36 附近）。

10.2 人如何编辑与批准计划：Cursor plan mode 演示

主讲人播放了 Cursor 的 plan mode 演示，这段演示把「计划作为人机接口」讲得很具体：

1. 用户提出一个较大改动（在 agents 页面做一个能看到队友后台 agent、并可以评论的看板）；
2. 模型建议切到 plan mode，然后**做深度信息收集**：搜索、读文件，弄清鉴权、授权、以及现有 UI 组件怎么用；
3. 模型带着**几个具体问题**回来问用户（新 tab 和路由怎么做、看板按状态分组还是卡片式、评论存哪里）；
4. 用户回答问题后，模型给出完整计划（Markdown 文件）：新的前端数据模型、新路由、待办列表；
5. 用户点「build」，整份计划被载入并分配给 agent；执行过程中可以看到待办项的进度；
6. 计划可以保存到工作区，作为团队协作的共享文档。

这段演示背后有一个很实用的设计：plan mode 在 Cursor 里被当作一种**权限模式**（permission mode）——它允许 agent 先做只读的信息收集动作，等信息足够、计划确认之后，再放开写操作。主讲人也在课程里把 plan mode 称为「人机共同思考的草稿纸」：它既让 agent 少走弯路，也让人在风险变高之前有一个可审阅、可编辑的介入点。

10.3 长程电脑操作的评测：Odysseus

计划与人机接口要做实验，就需要长程评测。主讲人介绍了他们最近发布的 benchmark **Odysseus**：基于人们**主动捐赠**的真实浏览历史（可以自行选择捐赠哪些），构造出诸如「帮我在我所在城市找 ACL 手术医生」这样的任务。解决这类任务需要数百步操作，可能要花半小时到一小时在电脑上（例子里是 93 步，打开一堆网站、把信息填进表格），评测用的是 **rubric**（检查任务各组成部分是否都被满足），而不是单一的成功/失败。

这个 benchmark 的意义在于：它测的不是「一次问答」，而是**长程、结构化、有真实世界复杂度**的电脑操作；这种任务正是计划、重规划、多 agent 分工最能发挥作用的地方（后半节课的 MACU 就在它上面做实验）。

10.4 本章小结

计划既是安全边界（把策略写进代码、把不可信数据挡在策略之外），也是人机接口（可审阅、可编辑、可批准的中间产物；plan mode 同时是一种权限模式）。长程电脑操作任务需要新的评测方式（Odysseus 的 rubric 评测），它们是计划与多 agent 方法的试金石。

11 多 agent：管理者 + 子代理的结构与实证

本讲标题里还有「多 agent 协作」。主讲人没有铺开讲通信机制（那是后面的交互课），而是用 MACU（Multi-Agent Computer Use）把「管理者 + 并行子代理」的结构和实验证据讲清楚。

11.1 MACU 的结构：DAG + 调度器 + 重规划

图 **??** 展示了 MACU 的完整回路，读图顺序如下：

1. 用户任务进来，交给 **Manager**（一个强 LLM，比如较新的 Claude Sonnet），由它把任务分解成一个 **DAG**（有向无环图）：节点是子任务，边是依赖；
2. **调度器循环**：按依赖顺序，把当前可执行的子任务分发给多个 **CUA 子代理**（Computer-Use Agent；每个子代理运行在自己的 VM 里，形成一个虚拟机池），彼此并行；
3. 子代理完成后进入 **follow-up loop**：Manager 检查结果，判断该子任务是「完成」还是「需要继续/重做」；
4. 如果结果不对，Manager 可以**重规划**：修改图——重做某个节点、或者新增节点（对应开头 GPU 例子里「加入第五家厂商」）；
5. 循环直到整张图处理完毕，输出总结与交付物。

这个结构把本讲前面所有概念串起来了：DAG 是计划表示；并行是模块化；follow-up 是执行反馈；改图是重规划；Manager 与子代理分开是「规划器与执行器分离」。

11.2 实验证据：多 agent 在哪里有用

主讲人给了几组很好的消融（图 **??** 是其中最后一组）：

- **多 agent 的收益与任务类型强相关**：在 OSWorld 上从单 agent 到多 agent 有明显提升；在 Odysseys 这类长程任务上提升最大（一个长任务里子任务多、可并行，多 agent 的优势最明显）。
- **更强的 Manager 改善协调**：固定子代理、只换 Manager，成功率随 Manager 变强而上升。
- **更弱的 worker 被抬升更多**：图 **??** 是 OSWorld 的 36 个任务、Manager 固定为 Opus 4.6 的消融——单 agent vs MACU：Qwen3.6-27B 47.2 → 66.7（+19.5），Qwen3.6-35B-A3B 50.0 → 58.3（+8.3），Qwen3.5-9B 50.0 → 61.1（+11.1），Qwen3.5-4B 25.0 → 58.3（+33.3），Qwen3.5-2B 27.8 → 44.4（+16.6）。**越弱、越小的 worker，加上 Manager 之后提升越大**——因为难的那部分（怎么拆）被 Manager 承担了。
- **重规划确实有用**：限制 Manager 修改图的次数，只允许 0 次时成功率约 25%，允许修改后提升，允许更多重规划后进一步提升——这是「重规划有用」的直接实验证据（也是本讲反复强调的结论的落脚点）。

11.3 这对系统设计意味着什么

把上面的证据合起来，可以得出一组非常实用的设计结论（主讲人在结尾也反复强调）：

1. **多 agent 不是免费的**：Manager 调用更强模型、调度与 VM 有成本、系统的可预测性下降；
2. **它的收益主要来自「分工」而非「讨论」**：把任务拆成可并行的子任务、把贵模型集中在规划上、把便宜模型用于执行；
3. **重规划是长程任务的关键能力**，允许修改计划的次数越多，收益越明显——但代价是更不可预测、更难被人监督；
4. **是否采用多 agent，取决于任务本身是否有清晰可分的子任务**，以及是否值得为它付出额外复杂度。

11.4 本章小结

MACU 用 Manager + DAG + 调度器 + CUA 子代理把「计划—并行执行—检查—重规划」做成闭环。实证显示：多 agent 的收益与任务类型强相关、更强 Manager 改善协调、更弱 worker 被抬升更多、允许更多重规划带来更高成功率。多 agent 的收益来自分工与并行，不是来自代理之间聊天。

12 总结与延伸

12.1 主讲人的收尾

主讲人最后把整讲落回一个判断：**加不加规划是一个设计选择，它明显增加复杂度，而这份复杂度不一定值得**。他给了一串应该先问自己的问题，我把它整理成一张检查表：

规划设计的检查清单（据主讲人收尾整理）

1. 任务里有没有清晰的子任务？子任务之间可分离吗？可以独立并行执行吗？
2. 有没有哪个子任务值得配一个专门的模型（更便宜/更专的模型）？
3. 子任务类型是否不同、需要不同的处理方式？
4. 规划本身比执行更难吗？（如果执行才是瓶颈，做计划可能没用）
5. 什么时候该重规划？该不该坚持原计划？改变计划带来的收益，值得它增加的复杂度吗？
6. 计划能不能成为控制手段：让人来监督/干预，或用另一个模型来引导？

12.2 把本讲压缩成六句话

本讲核心结论（背这几句）

1. 计划 = 对未来行为的显式表示（动作/子目标 + 顺序/依赖），是提案而非契约；核心价值是用结构换可预测性、模块化与可介入。
2. 经典规划靠显式表示与搜索保证可行性；LLM 规划靠先验直接生成计划，重心转为**检查可行性**与**根据反馈重规划**。
3. 计划的表示有谱系：形式化（可检查）→ 程序（可运行、可审计）→ 自然语言（最通用、可被人编辑）。
4. 执行反馈是关键：affordance 把「能不能做」纳入打分；相同 token 下多交互通常优于空想；过度思考显著伤害表现。
5. 长程任务受精度乘法（p^n）与上下文中毒双重约束；子任务越简单、上下文越干净越好。
6. 多 agent 的收益来自分工与并行：Manager 负责拆解与协调，弱 worker 被抬升更多，重规划次数增加带来更高成功率，但代价是成本与不可预测性。

12.3 延伸思考

1. **规划本质上是一种「人类可读的上下文压缩」**。前面讲上下文管理时，压缩是把历史压成摘要；计划则是把「接下来要做什么」压成一份可读、可编辑的产物。两者共享同一个动机：让 agent 在长任务里不被越来越多 token 淹没。差别在于计划是可被人审阅的压缩。

2. **计划是权限设计的载体**。plan mode 把计划当作一种权限模式：先只读收集信息，计划确认后再放开写权限。这个思路可推广到一切「让模型接触真实系统」的场景——把危险能力绑在计划批准之后，而不是绑在每一次动作上。

3. **训练视角：委托（delegation）是可以被训练的动作**。RAO 把「启动子代理」变成动作空间里的一项，奖励分解到树节点。这提示我们：agent 的 RL 训练里，**动作空间不只包括工具调用，还包括「怎么组织工作」这类元动作**（拆解、委托、重规划）。

4. **评测视角：长程任务需要 rubric 与消融**。Odysseus 用 rubric 拆解任务，MACU 用「允许改图几次」做消融——这两件事说明长程 agent 的评测不能只有一个端到端成功率，否则无法区

分「计划差」「执行差」「重规划差」。

5. **实践顺序**。如果要在业务里做 agent，建议先按检查清单判断是否真需要规划；需要的话，先做 plan mode（人可审阅）再考虑多 agent；多 agent 之前先做消融，确认收益来自分工而不是来自「更多调用」。

12.4 拓展阅读（课程中提到的文献，供进一步查阅）

- Kojima et al., *Large Language Models are Zero-Shot Reasoners*（「Let's think step by step」, 2022）。
- Zhou et al., *Least-to-Most Prompting Enables Complex Reasoning in Large Language Models* (ICLR)。
- Khot et al., *Decomposed Prompting*（分解 + 子任务类型 + 程序化控制）。
- SayCan：用价值函数给 LLM 生成的子任务做 affordance 重打分。
- ReAct、Reflexion、AlfWorld：交互式 agent 与基于反馈的重规划的基础工作。
- *Building Effective Agents*（Anthropic 博客，workflow 与 agent 的分界）。
- Plan-and-Act、Agentless：规划器/执行器分离与固定工作流的两条代表路线。
- *Vending-Bench*：长程商业模拟评测；*Odysseus*：基于真实浏览历史的长程电脑操作 benchmark。
- RAO（Recursive Agent Optimization）与 MACU（Multi-Agent Computer Use）：可训练的递归分解与管理者-子代理结构。